

Market-Basket Analysis on IMDB Actors

Stefania Lavarda

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

The aim of this project is to implement a system for frequent itemset mining, also known as market-basket analysis. The task is based on the IMDB Movies Dataset from Kaggle. In this project, each movie is considered as a basket, while the actors listed in the columns **Star1**, **Star2**, **Star3**, and **Star4** are considered as items.

The objective is to discover frequent combinations of actors that appear together in the same movies. After finding frequent itemsets, association rules are generated in order to describe relationships between actors.

The main algorithm used in this project is the A-Priori algorithm.

2 Dataset

The dataset used is the *IMDB Dataset of Top 1000 Movies and TV Shows*, available on Kaggle. The dataset is downloaded directly in the notebook using the Kaggle API, in order to make the experiment replicable. The Kaggle dataset identifier used in the code is:

`harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows`

The dataset contains information about 1000 movies and TV shows. Even if the dataset contains several columns, such as title, genre, year, rating, director, and gross income, only the actor columns are used for the market-basket analysis. The selected columns are:

- **Star1**
- **Star2**
- **Star3**
- **Star4**

These columns were chosen because the project requires each movie to be treated as a basket and the actors as items.

3 Data organization

The data was organized according to the market-basket model. In this model, a basket is a set of items. For this project:

- each movie is one basket;
- each actor is one item;
- each basket contains up to four actors.

The final list of transactions is therefore a list of actor baskets, where each row corresponds to one movie.

The dataset used in the experiment has the following characteristics:

Property	Value
Number of movies	1000
Number of baskets	1000
Number of unique actors	2709
Average basket size	3.996

Table 1: Summary of the dataset after basket creation.

4 Pre-processing

The pre-processing phase was simple because the dataset was already structured. The following operations were applied:

- only the columns `Star1`, `Star2`, `Star3`, and `Star4` were selected;
- rows with all actor values missing were removed;
- missing actor values inside a row were ignored;
- actor names were converted to strings;
- extra spaces were removed using `strip()`;
- duplicate actors inside the same basket were removed using a set;
- actors inside each basket were sorted to keep a consistent representation.

This produced a clean list of transactions suitable for frequent itemset mining.

5 Methodological approach

The main method used in this project is the A-Priori algorithm. A-Priori finds frequent itemsets in an efficient way by using the results from smaller itemsets to decide which larger itemsets should be checked. The key idea behind A-Priori is the monotonicity property:

If an itemset is frequent, then all of its subsets must also be frequent.

This property allows the algorithm to reduce the number of candidate itemsets that need to be considered. For example, if the pair {Actor A, Actor B} is not frequent, then no triple containing both Actor A and Actor B can be frequent. Therefore, there is no need to count such triples.

In this project, the A-Priori algorithm works as follows:

1. Count all single actors.
2. Keep only actors whose support is at least the minimum support threshold.
3. Generate candidate pairs from frequent actors.
4. Count candidate pairs and keep only frequent pairs.
5. Generate candidate triples only if all their pairs are frequent.
6. Continue with larger itemsets until no more frequent itemsets are found.

Since each movie basket contains at most four actors, the largest possible itemset has size four. The algorithm stops naturally when no more frequent itemsets are found.

After the frequent itemsets are obtained, association rules are generated. An association rule has the form:

$$A \rightarrow B$$

where A is the antecedent and B is the consequent. In this project, this can be interpreted as: if the actors in A appear in a movie, then the actors in B are also likely to appear.

For each rule, the confidence is computed as:

$$confidence(A \rightarrow B) = \frac{support(A \cup B)}{support(A)}$$

The interest of a rule is computed as:

$$interest(A \rightarrow B) = confidence(A \rightarrow B) - support(B)$$

Confidence measures the fraction of baskets containing the antecedent that also contain the consequent, while interest compares the confidence with the general frequency of the consequent in the whole dataset, showing whether the consequent appears more often when the antecedent is present.

6 Implementation

The project was implemented in Python 3 using a Jupyter notebook executable in Google Colab. The notebook follows the complete workflow, from downloading the dataset to exporting the final results.

First, the dataset is downloaded from Kaggle and loaded into a dataframe. Then, only the four actor columns are selected. Each row is transformed into a transaction, so that every movie becomes a basket of actors. During this step, missing values are removed, actor names are cleaned, and duplicate actors inside the same basket are eliminated.

The implementation then computes some basic information about the prepared data, such as the number of baskets, the number of distinct actors, and the average basket size.

The minimum support threshold is defined as a fraction of the dataset size. In the final experiment, the value 0.005 is used, meaning that an itemset must appear in at least 0.5% of the movies.

The function `get_support_count()` counts how many baskets contain each candidate itemset. Instead of comparing every candidate with every basket, the function generates only the combinations that actually appear inside each basket.

The function `apriori()` implements the A-Priori algorithm and returns a dictionary where each key is a frequent itemset and each value is its support count. The frequent itemsets are then converted into a table with four main columns: the itemset, its size, its support count, and its relative support.

The notebook also generates association rules from the frequent itemsets. For each frequent itemset with at least two actors, the code creates association rules by dividing the itemset into two parts: the antecedent, which is the left side of the rule, and the consequent, which is the right side of the rule. Then it computes support, confidence, and interest for each rule. Only rules whose confidence is at least 0.5 are kept. This means that the consequent must appear in at least 50% of the baskets that contain the antecedent. Rules with lower confidence are discarded because they are considered too weak.

Finally, the results are made more readable by converting actor tuples into strings. The notebook also produces bar charts for the most frequent single actors, actor pairs, and actor triples. The final tables of frequent itemsets and association rules are exported into CSV files, so the results can be reused in the report or checked separately.

7 Scalability

The proposed solution takes scalability into account, since the analysis should remain manageable when the dataset becomes larger. This is important because, in market-basket analysis, the number of possible item combinations can grow very quickly as the number of items increases.

First, the A-Priori algorithm reduces the number of itemsets that need to be analyzed. Instead of generating all possible actor combinations from the beginning, the algorithm builds candidates level by level. Larger candidates are generated only from itemsets that were already found to be frequent in the previous step.

Second, the implementation uses the monotonicity property to discard unnecessary candidates. If a smaller itemset is not frequent, then any larger itemset containing it cannot be frequent either.

Third, the minimum support is defined as a fraction of the dataset size:

$$\text{min_support} = \lceil \text{min_support_fraction} \times \text{number_of_baskets} \rceil$$

In the experiment, the selected value was:

$$\text{min_support_fraction} = 0.005$$

Since the dataset contains 1000 baskets, this corresponds to:

$$\text{min_support} = 5$$

Using a relative support threshold is better for scalability than using a fixed number. For example, if the dataset grows from 1,000 movies to 100,000 movies, a fixed threshold such as 3 would become too small and would produce too many rare itemsets. Instead, a relative threshold grows with the dataset size and keeps the analysis more controlled.

Finally, candidate support counting is done by generating only combinations that appear inside each basket. This avoids unnecessary subset checks against all candidates.

8 Experiments

Several support thresholds were tested in order to understand how the number of frequent itemsets changes. The tested values were relative thresholds, later converted into absolute support counts.

Minimum support fraction	Minimum support count	Frequent itemsets found
0.002	2	791
0.003	3	299
0.004	4	151
0.005	5	83
0.006	6	60
0.007	7	30
0.010	10	9

Table 2: Effect of the minimum support threshold on the number of frequent itemsets.

The final threshold chosen for the main analysis was 0.005, corresponding to a minimum support count of 5 movies. This value was selected because lower thresholds produced many itemsets, including many rare combinations, while higher thresholds produced too few results. The selected threshold gives a smaller and more interpretable set of frequent itemsets.

The final experiment produced 83 frequent itemsets:

Itemset size	Number of frequent itemsets
1	79
2	3
3	1
4	0

Table 3: Number of frequent itemsets by size using minimum support 5.

9 Results

The most frequent single actors were the actors appearing in the largest number of movies in the dataset. The top results include Robert De Niro, Tom Hanks, Al Pacino, Brad Pitt, and Clint Eastwood.

Actor	Support count
Robert De Niro	17
Tom Hanks	14
Al Pacino	13
Brad Pitt	12
Clint Eastwood	12
Christian Bale	11
Leonardo DiCaprio	11
Matt Damon	11
James Stewart	10
Michael Caine	9

Table 4: Top frequent single actors.

With the selected threshold $min_support_fraction = 0.005$, only three frequent actor pairs were found:

Actor pair	Support count	Support
Daniel Radcliffe, Rupert Grint	6	0.006
Daniel Radcliffe, Emma Watson	5	0.005
Emma Watson, Rupert Grint	5	0.005

Table 5: Frequent actor pairs.

One frequent triple was found:

Actor triple	Support count	Support
Daniel Radcliffe, Emma Watson, Rupert Grint	5	0.005

Table 6: Frequent actor triple.

The results are coherent with the dataset. Daniel Radcliffe, Emma Watson, and Rupert Grint appear together in several movies from the Harry Potter series. Therefore, their pair and triple combinations satisfy the selected minimum support threshold. The fact that only a few frequent pairs and one frequent triple are found shows that the threshold 0.005 is selective and keeps only the strongest actor co-occurrences.

10 Association rules

Association rules were generated from the frequent itemsets using a minimum confidence of 0.5.

Some of the strongest rules were:

Antecedent	Consequent	Confidence	Support count	Interest
Daniel Radcliffe	Rupert Grint	1.000	6	0.994
Rupert Grint	Daniel Radcliffe	1.000	6	0.994
Daniel Radcliffe, Emma Watson	Rupert Grint	1.000	5	0.994
Emma Watson, Rupert Grint	Daniel Radcliffe	1.000	5	0.994
Daniel Radcliffe	Emma Watson	0.833	5	0.826
Rupert Grint	Emma Watson	0.833	5	0.826
Daniel Radcliffe	Emma Watson, Rupert Grint	0.833	5	0.828
Rupert Grint	Daniel Radcliffe, Emma Watson	0.833	5	0.828
Daniel Radcliffe, Rupert Grint	Emma Watson	0.833	5	0.826
Emma Watson	Daniel Radcliffe	0.714	5	0.708

Table 7: Examples of association rules.

The rules with confidence equal to 1.0 indicate that, in this dataset, every movie containing the antecedent actor or actor group also contains the consequent actor. However, this should be interpreted carefully because the dataset contains only the top 1000 movies and only four actors per movie. Therefore, the rules are valid for this dataset representation, but they are not general facts about all movies.

The interest values are positive, which means that the rules are stronger than simply observing the general frequency of the consequent actor or actor group. Higher interest values indicate that the consequent appears much more often together with the antecedent than it does in the dataset overall.

11 Discussion

The results show that frequent itemset mining can discover meaningful actor collaborations. With the selected threshold, the strongest frequent combinations are related to the Harry Potter actors. This is reasonable because those actors appear together in multiple movies within the same film series.

The threshold selection has a strong effect on the results. When the minimum support count is 2, the algorithm finds 791 frequent itemsets. This is a large number and may include combinations that are not very meaningful. When the minimum support count is increased to 10, only 9 itemsets are found, which may be too restrictive. The chosen threshold of 5 gives 83 itemsets, which is easier to analyze.

One limitation of the project is that each basket contains only four actors, because only the columns `Star1`, `Star2`, `Star3`, and `Star4` are used. This means that the analysis ignores other actors who may also appear in the movie. Another limitation is that the dataset contains only 1000 movies, so the results are influenced by the composition of this specific dataset.

12 Conclusion

This project implemented a market-basket analysis system on the IMDB Movies Dataset. Movies were represented as baskets and actors were represented as items. After selecting and pre-processing the actor columns, the A-Priori algorithm was applied to discover frequent actor combinations.

The final analysis used a relative minimum support threshold of 0.005, corresponding to 5 movies, and a minimum confidence threshold of 0.5 for association rules. The algorithm found 83 frequent itemsets, including 79 single actors, 3 actor pairs, and 1 actor triple. The most relevant frequent combinations involved Daniel Radcliffe, Emma Watson, and Rupert Grint, which is consistent with their repeated collaboration in the Harry Potter movie series.

The solution takes scalability into account by using the A-Priori monotonicity property and a relative support threshold. In this way, the number of candidate itemsets is reduced, and the threshold adapts to the size of the dataset. The experiment is also replicable because the dataset is downloaded directly from Kaggle and all implementation steps are included in the notebook.

As future work, it would be useful to evaluate the manual Apriori implementation on larger datasets and compare its performance with alternative algorithms designed to improve scalability, such as the PCY algorithm. PCY improves scalability by using the main memory that remains unused during the first pass of A-Priori to store a hash table of bucket counts for pairs. Each pair generated from a basket is mapped to a bucket, and only the bucket count is increased. If a bucket count is below the support threshold, then no pair mapped to that bucket can be frequent. Therefore, in the second pass, PCY counts only pairs of frequent items that also hash to frequent buckets, reducing the number of candidate pairs and the memory needed for pair counting.

Overall, these results show that the Apriori algorithm find meaningful actor co-occurrence patterns in the IMDB dataset, while also highlighting the importance of choosing an appropriate support threshold and considering scalability in frequent itemset mining.